

# Technical Evaluation Report: Zone Boundary & Dynamic Route Corridor Visualization Tool

Project Title:

Zone Boundary + Dynamic Route  
Corridor Visualization Tool

## 1. Executive Summary & Objective

In the optimization of hyperlocal electric vehicle (EV) fleet networks, minimizing response latencies and maximizing batch-matching efficiency along active driver trajectories is highly critical. This report details the architecture and underlying algorithmic suite of an interactive, client-side web application designed for **Macro Rides**.

The system dynamically computes a flexible corridor around an active driver route, maps the complex physical topology onto a highly optimized discrete spatial grid, and screens available passenger pickup points for instant operational matching. To satisfy strict resource constraints and facilitate rapid evaluation, the solution is designed as a standalone frontend single-file architecture leveraging robust CDNs, ensuring absolute performance scaling without a server backend layer.

## 2. Comprehensive Breakdown of Core Tools

The spatial processing and frontend presentation layers integrate three specialized libraries, chosen to maximize mathematical execution velocity and prevent client rendering overhead.

| 1. Uber H3 Indexing                                                                                                                                                                                                                                             | 2. Turf.js Engine                                                                                                                                                                                                                       | 3. Leaflet.js Canvas                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A hexagonal hierarchical spatial index system that partitions the Earth's surface into discrete, uniform hexagonal cells. It converts traditional geographical coordinates (latitude and longitude pairs) into deterministic 64-bit hexadecimal string indexes. | An advanced geospatial processing library written natively in JavaScript. It executes client-side computational geometry—such as buffer creation, line interpolation, and polygon clipping—directly using standardized GeoJSON objects. | A lightweight, open-source mapping engine tailored for performance-critical vector data rendering. It bypasses heavy authentication and token frameworks, processing direct interactive geometric visualizations efficiently. |

## Deep-Dive into Uber H3 Algorithmic Advantages

Traditional geospatial calculations depend on continuous Euclidean or Haversine trigonometry to isolate intersections between a point feature and an irregular polygon. For a system tracking  $N$  assets against  $M$  pickup hotspots, this mandates an expensive  $O(N \times M)$  distance computation layout every clock cycle.

By utilizing H3, the geographic plane is quantized into discrete spatial units. The spatial containment problem degrades from continuous geometry to a hash-set inclusion test.

- **Hexagonal Geometry Invariance:** Unlike squares or triangles, hexagons maintain uniform spacing between their centroids and all adjacent neighbors. This minimizes orientation distortions when computing dynamic movement paths.
- **Constant-Time Complexity:** Converting coordinates to an H3 token occurs in  $O(1)$  time. Filtering pickup targets within an active corridor array resolves via rapid key checks inside a compiled JavaScript Set data structure, yielding true  $O(1)$  operational match complexity per point.
- **Configurable Scale Resolutions:** The engine exposes variable resolution scales. The application utilizes **Resolution 9** (edge length  $\sim 100m$ , area  $\sim 0.1\text{ km}^2$ ) as its baseline matching matrix, which perfectly fits the hyperlocal granularity requirements of a  $350m$  corridor buffer.

## 3. System Architecture & Data Pipeline

The runtime execution lifecycle functions as an asynchronous reactive loop, handling asset tracking updates and spatial filtering across four main pipeline phases:

| Pipeline Phase                  | Tool Responsibilities | Underlying Operation & Technical Output                                                                                                                              |
|---------------------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Geocoding & Teleportation    | OSM Nominatim API     | Converts user-typed address strings into coordinates via async Fetch requests, triggering a procedural route generator framework around the target center.           |
| 2. Vector Buffer Formulation    | Turf.js               | Processes the driver's active coordinate node and constructs a mathematically smooth polygon bounding box matching the configured metric radius input.               |
| 3. Spatial Quantization Mapping | H3-js Engine          | Translates the bounding buffer vertices into an array of H3 cell indexes via <code>polygonToCells()</code> , organizing them inside a high-speed runtime lookup Set. |
| 4. UI State Vector Rendering    | Leaflet.js            | Redraws the physical assets and tints pickup markers conditionally (Red = Eligible match inside index Set, Grey = Ineligible out-of-bounds node).                    |

### Core Mathematical Formula:

$$P_{\{eligible\}} = \{ p \in P \mid H3_{\{res\}}(p_{\{coords\}}) \in H3_{\{polygonToCells\}}(Buffer_{\{Tur\}}(d_{\{coords\}}, r)) \}$$

Where  $p$  represents a pickup point,  $P$  is the set of all active points,  $d_{\{coords\}}$  is the active driver position, and  $r$  is the user-configured corridor buffer radius.

## 4. Interactive Configuration & Control Layer Features

To allow complete auditing capabilities by the technical evaluation team, the application exposes multiple dynamic configuration inputs embedded within an integrated control dashboard interface:

1. **Global Address Geocoding Input:** Users type text queries (e.g., "*IIT Kanpur*", "*San Francisco Downtown*", "*New Delhi*"). The geocoding system relocates the operational environment, procedurally generates localized route trajectories, and redistributes simulation hotspots instantly.
2. **Variable Corridor Radius Slider:** Modifies the matching catchment threshold in real time from **100m** to **1000m**. Evaluators can visually verify how the H3 cell footprint scales horizontally.
3. **H3 Grid Resolution Selection:** Adjusts spatial cell allocation boundaries between Resolution 8 (**~700m**), Resolution 9 (**~100m**), and Resolution 10 (**~40m**) to showcase how structural grid precision balances CPU processing loads.
4. **Dynamic Pickup Density Adjuster:** Seeds between 5 and 150 independent passenger hotspot points via an uniform randomized distribution model, validating processing performance under heavy demand stress tests.

## Live Deployments

Interactive Production Demo: [macro-rides-mobility.netlify.app](https://macro-rides-mobility.netlify.app)

Source Code Repository: [github.com/shaswatgithub/macro-rides-mobility](https://github.com/shaswatgithub/macro-rides-mobility)